

宁波大学
NINGBO UNIVERSITY

云计算技术实验报告

[鲲鹏云容器]

姓名：	顾凯琪
学号：	226001624
课程名称：	云计算技术
所在学院：	信息科学与工程学院
专业班级：	22 级计算机科学与技术 2 班
任课教师：	顾凯琪

实验项目名称	**鲲鹏云容器**

一、实验目的与要求：

实验一：Docker 容器的基本操作

实验目的：

1. 掌握 Docker 环境的搭建：
    - 学习在华为云 ECS 实例上安装 Docker，配置依赖项（如 device-mapper、lvm2）。
    - 验证 Docker 服务启动状态及 hello-world 镜像的运行。
2. 熟练操作 Docker 镜像：
    - 通过 docker pull 从公共仓库拉取镜像(如 nginx、ubuntu:14.04)。
    - 使用 docker images 查询镜像列表，docker rmi 删除镜像（包括强制删除占用镜像）。
3. 管理容器生命周期：
    - 通过 docker create、docker start/stop/restart 管理容器启停。
    - 使用 docker exec 与 docker attach 进入容器交互界面，掌握后台运行容器（-d 参数）。
4. 搭建私有镜像仓库：
    - 部署本地 docker-registry 服务，通过 docker tag 和 docker push 实现镜像的本地化存储与共享。

实验要求：

1. 环境配置：
    - 创建华为云 VPC、安全组（放通 22、80、5000 端口），并购买鲲鹏架构 ECS 实例。
    - 安装 Docker 后，验证 docker run hello-world 输出成功信息。
2. 镜像操作验证：
    - 拉取并删除至少两个镜像（如 nginx、httpd），记录操作命令及结果。
3. 容器管理：
    - 创建并启动一个基于 httpd 镜像的容器，通过 docker ps 查看运

行状态。

- 使用 docker exec 进入容器内部，执行 ps 命令验证进程。

4. 私有仓库验证：
    - 将 ubuntu:14.04 镜像推送至本地仓库，并通过 curl 查询仓库镜像列表。

实验二：Dockerfile 的基本操作

实验目的：

1. 理解 Dockerfile 核心指令：
    - 掌握 FROM、ADD、RUN、EXPOSE、CMD 等指令的功能与应用场景。
    - 学习通过多阶段构建优化镜像大小（如编译环境与运行环境分离）。
2. 构建自定义镜像：
    - 编写 Dockerfile，从 centos:7 基础镜像编译安装 Nginx 1.12.2。
    - 配置 Nginx 运行参数（如用户权限、模块启用），并设置非守护进程模式。
3. 服务验证与优化：
    - 通过端口映射（-p 80:80）运行容器，验证浏览器访问 Welcome to Nginx 页面。
    - 优化 Dockerfile，添加 CMD 指令实现容器自动启动服务。

实验要求：

1. Dockerfile 编写：
    - 创建 nginx_demo 目录，下载 Nginx 源码包，编写 Dockerfile 完成编译安装。
    - 确保 Dockerfile 包含依赖安装（如 gcc、openssl-devel）及权限配置。
2. 镜像构建与验证：
    - 使用 docker build -t my_nginx:v1 构建镜像，记录构建日志中的关键步骤。

- 运行容器并映射端口至81，通过浏览器访问验证服务可用性。

3. 资源清理：
   - 实验结束后删除ECS实例及相关资源，确保无残留计费项。

实验三：鲲鹏平台OA系统容器化部署

实验目的：

1. 掌握企业应用容器化流程：
   - 通过Dockerfile将Java应用（OA系统）打包为镜像，包含运行时环境（JDK 8）及依赖项。
   - 配置镜像加速器，优化镜像拉取速度。
2. 华为云服务集成：
   - 使用SWR（容器镜像服务）托管自定义镜像，掌握docker login、docker tag、docker push操作。
   - 在CCE（云容器引擎）中创建鲲鹏集群，部署有状态工作负载（StatefulSet）。
3. 应用运维与监控：
   - 通过CCE控制台查看容器日志，监控服务运行状态。
   - 实现弹性公网IP绑定，通过浏览器访问OA系统登录页。

实验要求：

1. 镜像构建与上传：
   - 编写Dockerfile，基于arm64v8/adoptopenjdk镜像添加OA系统JAR包，暴露8088端口。
   - 将镜像推送至SWR指定组织，确保命名符合规范（如swr.cn-north-4.myhuaweicloud.com/组织名/镜像名:标签）。
2. CCE集群部署：
   - 创建鲲鹏集群（3控制节点，1工作节点），配置容器隧道网络与安全组策略。
   - 部署OA系统工作负载，设置服务类型为LoadBalancer，映射8088端口至公网。

3. 验证与清理：

- 使用弹性 IP+端口访问 OA 系统，以用户名 soli 和密码 123456 登录验证功能。
- 实验结束后释放 CCE 集群、云数据库 GaussDB 及 ECS 实例，提交资源释放截图。

二、实验环境：

设备名称	设备型号	软件版本
鲲鹏平台OA系统服务器 ecs-oa	鲲鹏计算，通用计算增强型，kc1.2xlarge.2 | 8vCPUs | 16GB	openEuler 20.03 64bit with ARM
云数据库 GaussDB(for MySQL) gauss-oa	GaussDB(for MySQL)，通用增强型，16vCPUs | 64 GB	兼容MySQL 8.0

实验设备详细信息

设备名称	设备型号	软件版本
鲲鹏平台MySQL服务器 ecs-mysql	鲲鹏计算，通用计算增强型， kc1.large.2 | 2vCPUs | 4GB	openEuler 20.03 64bit
鲲鹏平台OA系统服务器 ecs-oa	鲲鹏计算，通用计算增强型， kc1.2xlarge.2 | 8vCPUs | 16GB	openEuler 20.03 64bit with ARM
云数据库GaussDB(for MySQL)gauss-oa	GaussDB(for MySQL)，通用增强型，16vCPUs | 64 GB	兼容MySQL 8.0

实验工具详细信息

名称	下载链接	用途
PuTTY	win64 位： https://the.earth.li/~sgtatham/putty/latest/w64/putty-64bit-0.74-installer.msi	远程登录工具
WinSCP	https://winscp.net/eng/download.php	远程传输工具

Git	https://git-scm.com/downloads	分布式版本控制系统
Maven	https://mirrors.huaweicloud.com/apache/maven/maven-3/3.6.3/binaries/apache-maven-3.6.3-bin.tar.gz	软件项目管理工具
oasys_mysql	https://gitee.com/github-5407963/oasys_mysql	OA系统项目源代码
oasys_mysql.sql	https://obs-oa-zyl.obs.cn-north-4.myhuaweicloud.com/oasys_mysql.sql	OA系统MySQL数据库脚本

三、实验方法（原理、流程图）

1. 实验原理
   - 容器化核心：

     Docker 利用 Linux 内核的 Cgroups 和 Namespace 实现资源隔离，镜像通过分层存储（UnionFS）减少冗余。
   - Dockerfile 构建逻辑：

     指令逐层执行，每层生成只读文件系统，最终镜像包含完整应用环境（如 Nginx 编译环境+运行配置）。
   - 华为云服务联动：

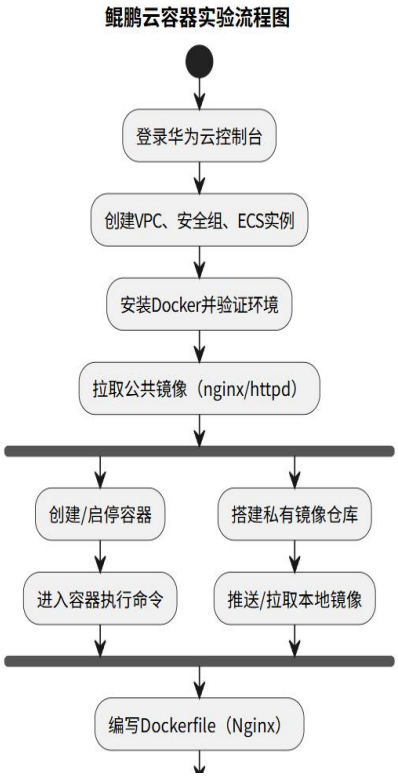
     SWR 提供私有镜像仓库，CCE 通过 Kubernetes 编排容器，实现应用的高可用与弹性伸缩。
2. 关键步骤详解：
   1. 实验一：
      - 通过 yum install docker 安装 Docker，配置安全组放通 5000 端口（私有仓库）。
      - 使用 docker run -d -p 5000:5000 --name registry registry 启动本地仓库，验证镜像推送/拉取。
   2. 实验二：
      - 在 Dockerfile 中通过 RUN ./configure --user=nginx 指定编译

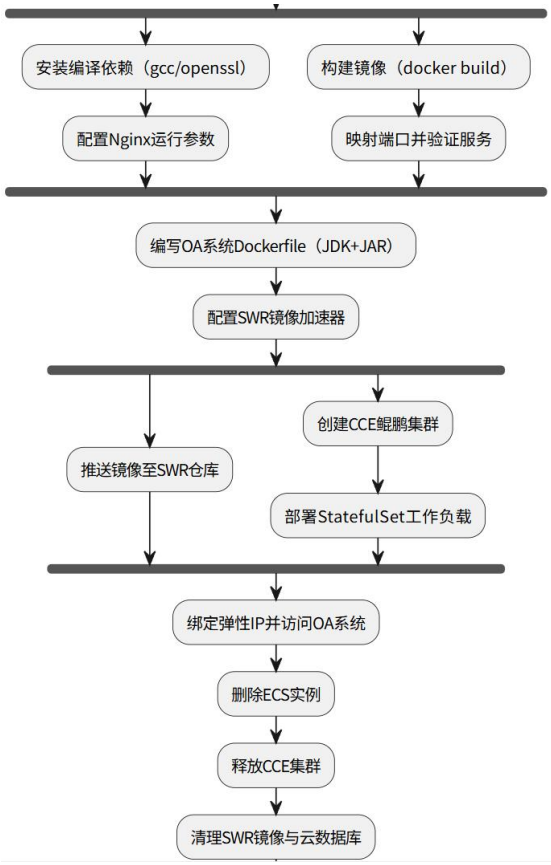
参数，EXPOSE 80 声明服务端口。

- 运行容器时添加-g "daemon off;"保持前台进程，避免容器自动退出。

3. 实验三：
   - 在 CCE 中创建 StatefulSet 时，需绑定 SWR 镜像地址，配置服务端口映射（8088:8088）。
   - 通过华为云 ELB（弹性负载均衡）分配公网 IP，实现外部访问 OA 系统。

3. 实验流程图





四、实验过程、步骤及内容

目录

实验一 Docker 容器的基本操作 8

一. Docker 的安装和配置 12

二.镜像的基本操作 14

三.容器的基本操作 15

四．私有仓库的搭建 19

实验二 Dockerfile 的基本操作 21

五．Dockerfile 文件构建 21

六. Nginx 镜像验证 23

实验三 鲲鹏平台 OA 系统容器化部署 26

七．通过 Dockerfile 构建 OA 系统镜像 26

八．CCE 鲲鹏集群创建 32

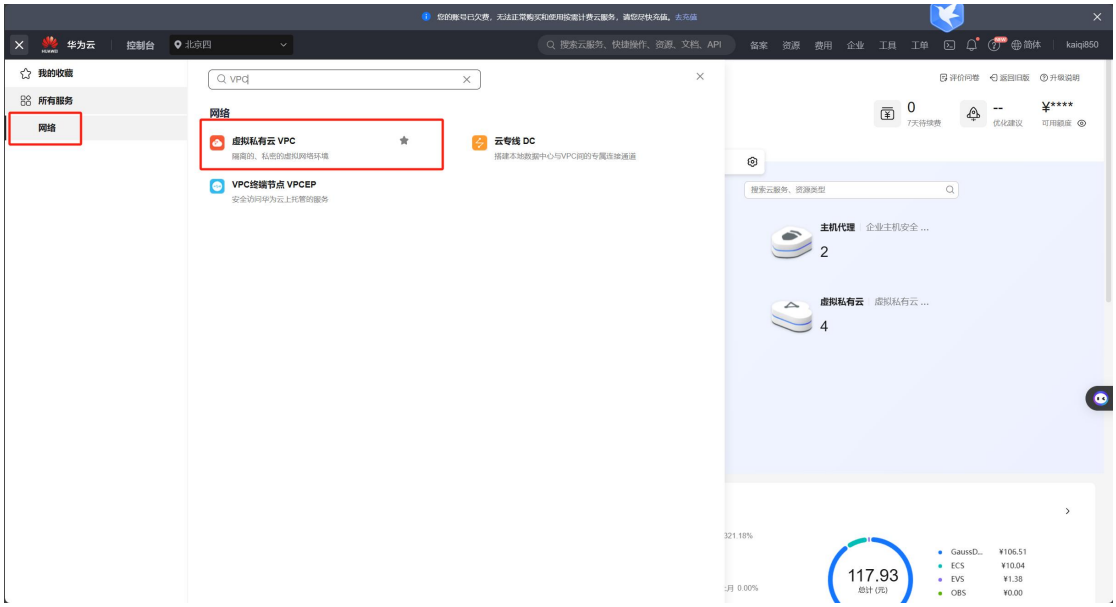
九．CCE 部署 OA 系统 33

十．释放资源 36

问题与解决: 38

实验一 Docker 容器的基本操作

创建虚拟私有云



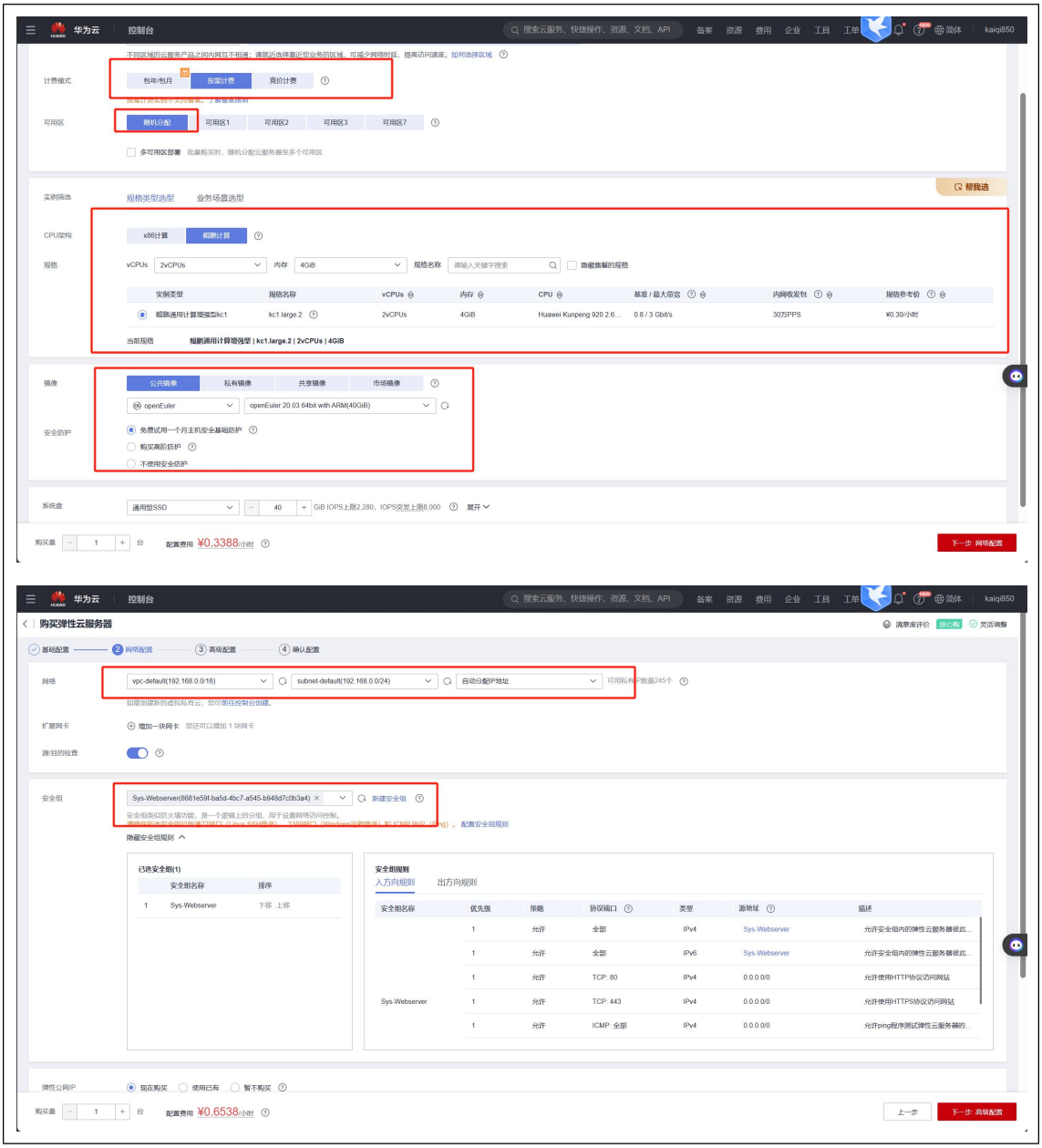
沿用上一个实验的 VPC



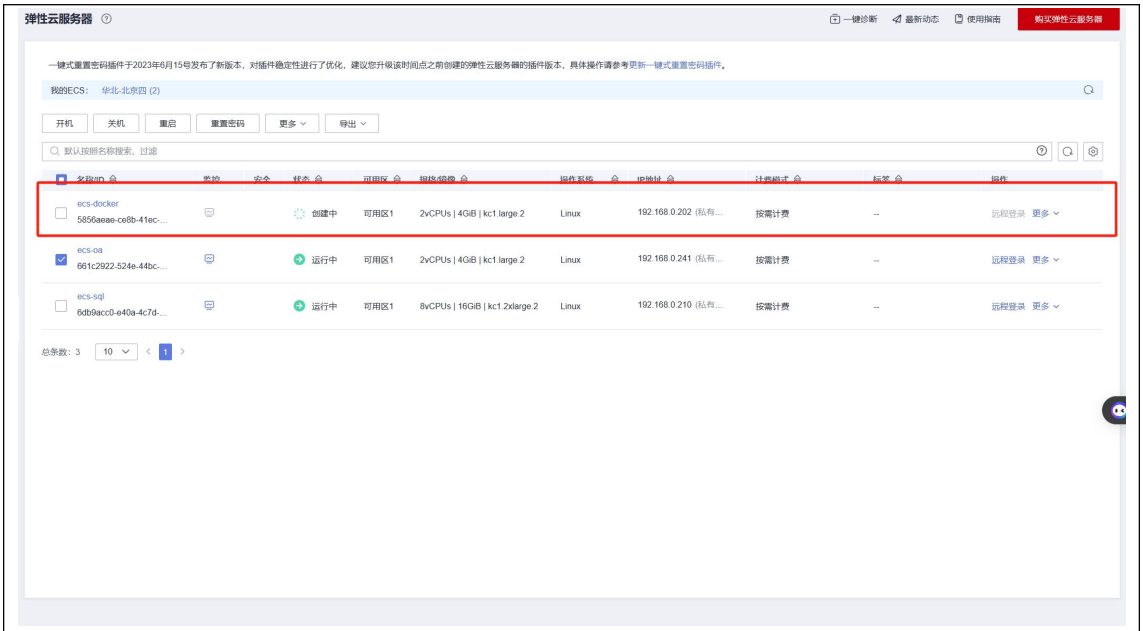
创建并配置安全组，在“创建安全组”窗口中配置模板选择为“通用 Web 服务器”，名称设置为“Sys-Webserver”，然后点击“确定”。默认放通 22、3389、80、443 端口和 ICMP 协议。



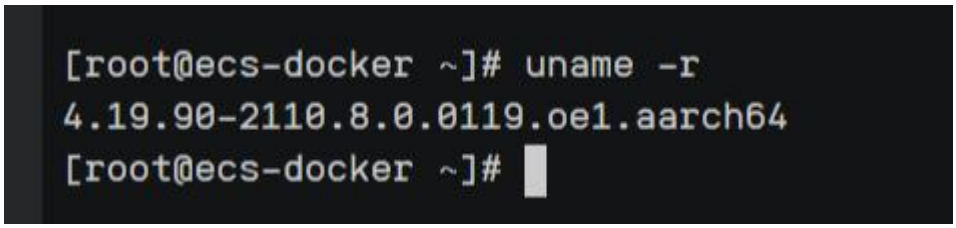
购买弹性云服务器


华为云
控制台
搜索云服务、快捷操作、资源、文档、API
备案
资源
费用
企业
工具
工单
简体
kaiqi850
计费模式
包年/包月
按需计费
竞价计费
可用区
随机分配
可用区1
可用区2
可用区3
可用区7
多可用区部署
批量购买时，随机分配云服务器至多个可用区
实例筛选
规格类型选型
业务场景选型
帮我选
CPU架构
x86计算
鲲鹏计算
规格
vCPUs
2vCPUs
内存
4GiB
规格名称
请输入关键字搜索
隐藏售罄的规格
实例类型
规格名称
vCPUs
内存
CPU
基准 / 最大带宽
内网收发包
规格参考价
鲲鹏通用计算增强型kc1
kc1.large.2
2vCPUs
4GiB
Huawei Kunpeng 920 2.6...
0.8 / 3 Gbit/s
30万PPS
¥0.30/小时
当前规格
鲲鹏通用计算增强型 | kc1.large.2 | 2vCPUs | 4GiB
镜像
公共镜像
私有镜像
共享镜像
市场镜像
openEuler
openEuler 20.03 64bit with ARM(40GiB)
安全防护
免费试用一个月主机安全基础防护
购买高阶防护
不使用安全防护
系统盘
通用型SSD
40
GiB IOPS上限2,280，IOPS突发上限8,000
展开
购买量
1
台
配置费用
¥0.3388/小时
下一步 网络配置

华为云
控制台
搜索云服务、快捷操作、资源、文档、API
备案
资源
费用
企业
工具
工单
简体
kaiqi850
购买弹性云服务器
满意度评价
放心购
灵活调整
基础配置
网络配置
高级配置
确认配置
网络
vpc-default(192.168.0.0/16)
subnet-default(192.168.0.0/24)
自动分配IP地址
可用私有IP数量245个
如需创建新的虚拟私有云，您可前往控制台创建。
扩展网卡
增加一块网卡
您还可以增加 1 块网卡
源/目的检查
安全组
Sys-Webserver(8681e59f-ba5d-4bc7-a545-b948d7c0b3a4)
新建安全组
安全组类似防火墙功能，是一个逻辑上的分组，用于设置网络访问控制。
配置安全组规则
隐藏安全组规则
已选安全组(1)
安全组名称
排序
1
Sys-Webserver
下移 上移
安全组规则
入方向规则
出方向规则
安全组名称
优先级
策略
协议端口
类型
源地址
描述
1
允许
全部
IPv4
Sys-Webserver
允许安全组内的弹性云服务器彼此...
1
允许
全部
IPv6
Sys-Webserver
允许安全组内的弹性云服务器彼此...
1
允许
TCP: 80
IPv4
0.0.0.0/0
允许使用HTTP协议访问网站
Sys-Webserver
1
允许
TCP: 443
IPv4
0.0.0.0/0
允许使用HTTPS协议访问网站
1
允许
ICMP: 全部
IPv4
0.0.0.0/0
允许ping程序测试弹性云服务器的...
弹性公网IP
现在购买
使用已有
暂不购买
购买量
1
台
配置费用
¥0.6538/小时
上一步
下一步 高级配置



使用 cloudshell 远程登陆，并检查内核版本



移除旧的版本

```
[root@ecs-docker ~]# uname -r
4.19.90-2110.8.0.0119.oe1.aarch64
[root@ecs-docker ~]# yum remove docker docker-client docker-client-latest docker-common docker-latest docker-latest-logrotate docker-logrotate docker-selinux docker-engine
-selinux docker-engine
No match for argument: docker
No match for argument: docker-client
No match for argument: docker-client-latest
No match for argument: docker-common
No match for argument: docker-latest
No match for argument: docker-latest-logrotate
No match for argument: docker-logrotate
No match for argument: docker-selinux
No match for argument: docker-engine-selinux
No match for argument: docker-engine
No packages marked for removal.
Dependencies resolved.
Nothing to do.
Complete!
[root@ecs-docker ~]#
```

安装 Docker 依赖工具

```
================================================================================
Install  4 Packages

Total download size: 1.9 M
Installed size: 8.0 M
Downloading Packages:
(1/4): device-mapper-event-1.02.170-3.oe1.aarch64.rpm                1.0 MB/s |  42 kB     00:00
(2/4): libaio-0.3.112-1.oe1.aarch64.rpm                              441 kB/s |  22 kB     00:00
(3/4): thin-provisioning-tools-0.7.6-6.oe1.aarch64.rpm                13 MB/s | 353 kB     00:00
(4/4): lvm2-2.02.187-3.oe1.aarch64.rpm                                19 MB/s | 1.5 MB     00:00
--------------------------------------------------------------------------------
Total                                                                 23 MB/s | 1.9 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                              1/1
  Installing       : libaio-0.3.112-1.oe1.aarch64                                 1/4
  Installing       : thin-provisioning-tools-0.7.6-6.oe1.aarch64                  2/4
  Installing       : device-mapper-event-8:1.02.170-3.oe1.aarch64                 3/4
  Running scriptlet: device-mapper-event-8:1.02.170-3.oe1.aarch64                 3/4
Created symlink /etc/systemd/system/sockets.target.wants/dm-event.socket → /usr/lib/systemd/system/dm-event.socket.

  Installing       : lvm2-8:2.02.187-3.oe1.aarch64                                4/4
  Running scriptlet: lvm2-8:2.02.187-3.oe1.aarch64                                4/4
Created symlink /etc/systemd/system/sysinit.target.wants/lvm2-monitor.service → /usr/lib/systemd/system/lvm2-monitor.service.
Created symlink /etc/systemd/system/sysinit.target.wants/lvm2-lvmetad.socket → /usr/lib/systemd/system/lvm2-lvmetad.socket.
Created symlink /etc/systemd/system/sysinit.target.wants/lvm2-lvmpolld.socket → /usr/lib/systemd/system/lvm2-lvmpolld.socket.

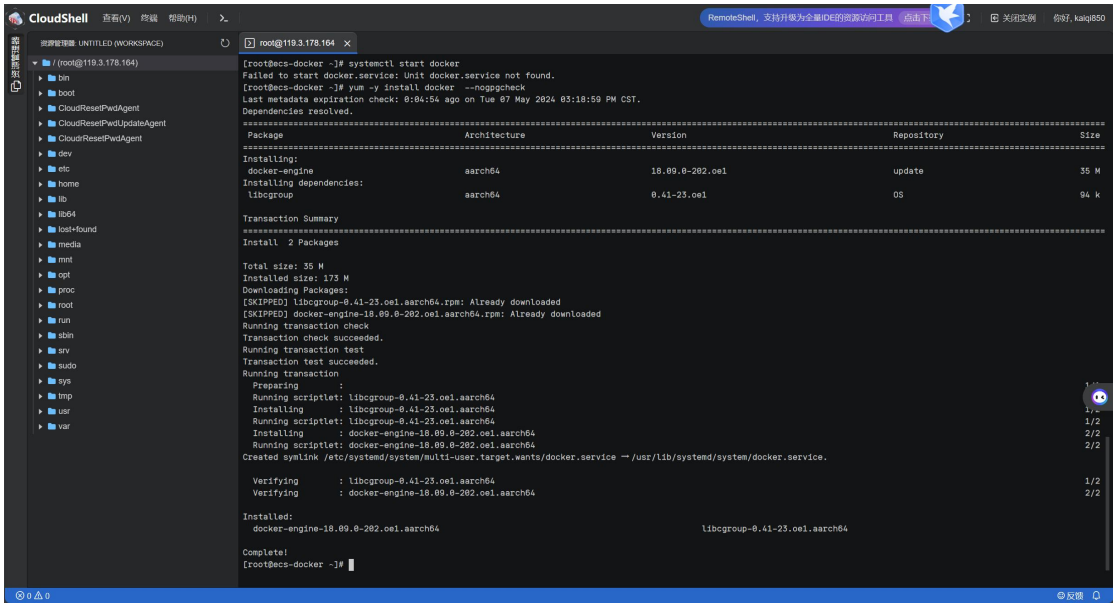
  Verifying        : device-mapper-event-8:1.02.170-3.oe1.aarch64                 1/4
  Verifying        : libaio-0.3.112-1.oe1.aarch64                                 2/4
  Verifying        : lvm2-8:2.02.187-3.oe1.aarch64                                3/4
  Verifying        : thin-provisioning-tools-0.7.6-6.oe1.aarch64                  4/4

Installed:
  lvm2-8:2.02.187-3.oe1.aarch64      thin-provisioning-tools-0.7.6-6.oe1.aarch64      device-mapper-event-8:1.02.170-3.oe1.aarch64      libaio-0.3.112-1.oe1.aarch64

Complete!
```

一. Docker 的安装和配置

输入命令 yum -y install docker，安装 docker。



启动 Docker 后台服务

```
[root@ecs-docker ~]# systemctl start docker
[root@ecs-docker ~]#
```

测试运行 hello-world

```
[root@ecs-docker ~]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
478afc919002: Pull complete
Digest: sha256:a26bff933ddc26d5cdf7faa98b4ae1e3ec20c4985e6f87ac0973052224d24302
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

由于本地没有 hello-world 这个镜像，所以会下载一个 hello-world 的镜像，并在容器内运行。若能正常输出信息，则说明安装成功。

查看下载的 hello-world 镜像。

```
[root@ecs-docker ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
hello-world         latest              ee301c921b8a        12 months ago       9.14kB
[root@ecs-docker ~]#
```

二.镜像的基本操作

下载 nginx 镜像

```
[root@ecs-docker ~]#  docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
22d97f6a5d13: Pull complete
a68473e9d216: Pull complete
7c0a9e5032ac: Pull complete
43f7d7c8e1e1: Pull complete
a23342af148c: Pull complete
18b7abeaeadc: Pull complete
568b5ecc128d: Pull complete
Digest: sha256:32e76d4f34f80e479964a0fbd4c5b4f6967b5322c8d004e9cf0cb81c93510766
Status: Downloaded newer image for nginx:latest
[root@ecs-docker ~]#
```

查询及删除镜像，使用 docker images 查看当前镜像

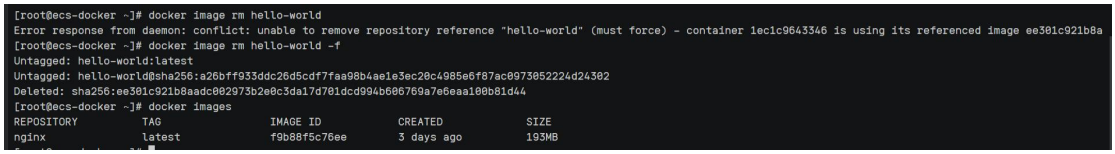
```
[root@ecs-docker ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx               latest              f9b88f5c76ee        3 days ago          193MB
hello-world         latest              ee301c921b8a        12 months ago       9.14kB
[root@ecs-docker ~]#
```

查看镜像的大小

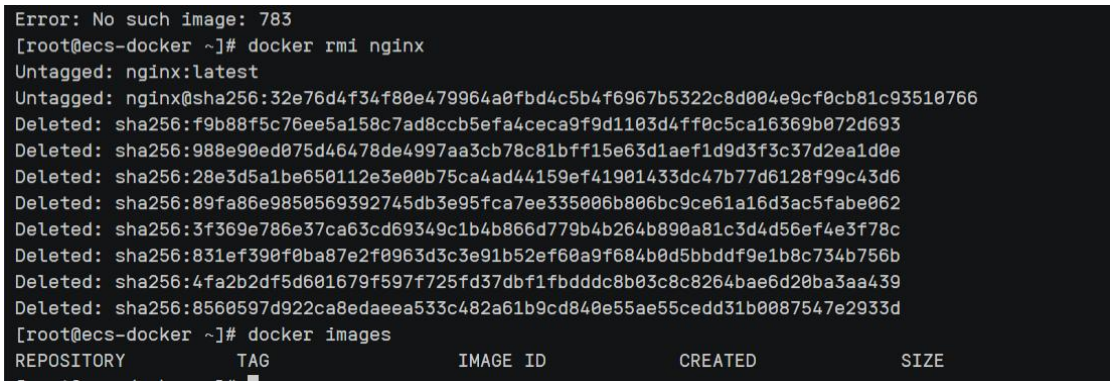
```
[root@ecs-docker ~]# docker system df
TYPE                TOTAL               ACTIVE              SIZE                RECLAIMABLE
Images              2                   1                   192.9MB             192.9MB (99%)
Containers          1                   0                   0B                  0B
Local Volumes       0                   0                   0B                  0B
Build Cache         0                   0                   0B                  0B
```

删除镜像：

-f ：强制删除，一般用于被占用的删除。观察到 hello world 原先无法删除，后使用-f 强制被删除



通过短 ID 或镜像完整 ID 删除镜像。



三.容器的基本操作

容器的创建与启停

创建一个基于 httpd 镜像的新容器。若主机中没有对应镜像，将会从 docker Hub 中拉取最新镜像。



查看容器信息，docker ps 命令表示列出所有在运行的容器信息；docker ps -a 命令显示所有的容器，包括未运行的。

```
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                     PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  38 seconds ago      Created                                        keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            11 minutes ago      Exited (0) 11 minutes ago                      cocky_ramanujan
[root@ecs-docker ~]#
```

可以查询到容器 ID 为 b79e6fc69cab，容器名称为 keen_sammet，容器状态为已创建 Created。

根据显示的容器 ID 或容器名称启动容器。

```
[root@ecs-docker ~]#  docker start b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                     PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  3 minutes ago       Up 8 seconds               80/tcp              keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            14 minutes ago      Exited (0) 14 minutes ago                      cocky_ramanujan
```

显示状态为 up，且有了 ports

停止容器运行

```
[root@ecs-docker ~]# docker stop b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                     PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  5 minutes ago       Exited (0) 6 seconds ago                       keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            15 minutes ago      Exited (0) 15 minutes ago                      cocky_ramanujan
```

重启容器

```
[root@ecs-docker ~]# docker restart b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                     PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  5 minutes ago       Up 5 seconds               80/tcp              keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            16 minutes ago      Exited (0) 16 minutes ago                      cocky_ramanujan
```

暂停容器

```
[root@ecs-docker ~]# docker pause b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                     PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  6 minutes ago       Up 50 seconds (Paused)     80/tcp              keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            17 minutes ago      Exited (0) 17 minutes ago                      cocky_ramanujan
```

恢复暂停的容器

```
[root@ecs-docker ~]# docker unpause b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  7 minutes ago       Up About a minute           80/tcp              keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            18 minutes ago      Exited (0) 18 minutes ago                       cocky_ramanujan
```

杀掉容器进程，强制停止容器

```
[root@ecs-docker ~]# docker kill b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  8 minutes ago       Exited (137) 3 seconds ago                      keen_sammet
1ec1c9643346        ee301c921b8a        "/hello"            19 minutes ago      Exited (0) 19 minutes ago                       cocky_ramanujan
```

启动容器，给容器重新命名

```
[root@ecs-docker ~]# docker start b79e6fc69cab
b79e6fc69cab
[root@ecs-docker ~]# docker rename b79e6fc69cab myhttpd
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS                      PORTS               NAMES
b79e6fc69cab        httpd               "httpd-foreground"  9 minutes ago       Up 21 seconds               80/tcp              myhttpd
1ec1c9643346        ee301c921b8a        "/hello"            20 minutes ago      Exited (0) 20 minutes ago                       cocky_ramanujan
```

容器的运行

运行一个新容器，该容器基于 ubuntu:14.04。若主机中没有对应镜像，将会从 docker Hub 中拉取最新镜像

```
[root@ecs-docker ~]# docker run ubuntu:14.04 /bin/echo 'Hello world'
Unable to find image 'ubuntu:14.04' locally
14.04: Pulling from library/ubuntu
d1a5a1e51f25: Pull complete
75f8eea31a63: Pull complete
a72d031efbfb: Pull complete
Digest: sha256:64483f3496c1373bfd55348e88694d1c4d0c9b660dee6bfef5e12f43b9933b30
Status: Downloaded newer image for ubuntu:14.04
Hello world
```

下面的命令则启动一个 bash 终端，允许用户进行交互

```
[root@ecs-docker ~]# docker run -it ubuntu:14.04 /bin/bash
root@a40de24c091b:/# pwd
/
root@a40de24c091b:/# ls
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

更多的时候，需要让 Docker 在后台运行而不是直接把执行命令的结果输出在当前宿主机下。此时，可以通过添加-d 参数来实现。下面看下两者的区别：

```
[root@ecs-docker ~]# docker run ubuntu:14.04 /bin/sh -c "while true; do echo hello world; sleep 1; done"
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
```

通过 docker logs [container ID or NAMES]获取容器的输出信息。

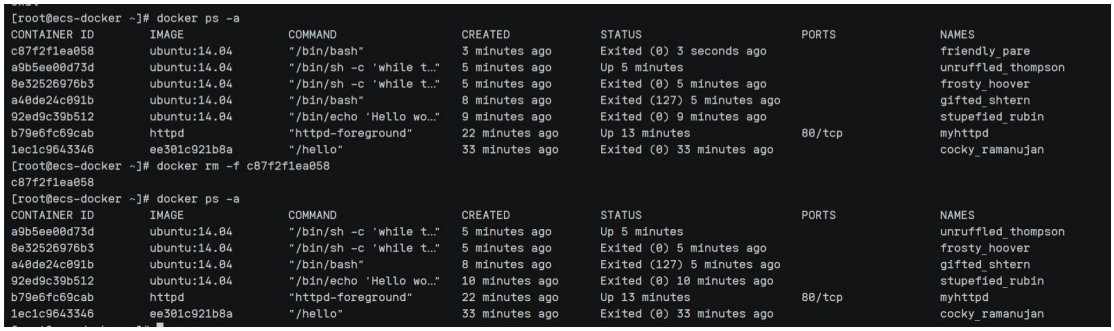
```
[root@ecs-docker ~]# docker run -d ubuntu:14.04 /bin/sh -c "while true; do echo hello world; sleep 1; done"
a9b5ee00d73d947538b2d89aa244d820bc55c4549592079d7d957f971433235c
[root@ecs-docker ~]# docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS                          PORTS               NAMES
a9b5ee00d73d        ubuntu:14.04        "/bin/sh -c 'while t…"   34 seconds ago      Up 33 seconds                                       unruffled_thompson
8e32526976b3        ubuntu:14.04        "/bin/sh -c 'while t…"   About a minute ago  Exited (0) 50 seconds ago                           frosty_hoover
a40de24c091b        ubuntu:14.04        "/bin/bash"              3 minutes ago       Exited (127) About a minute ago                     gifted_shtern
92ed9c39b512        ubuntu:14.04        "/bin/echo 'Hello wo…"   5 minutes ago       Exited (0) 5 minutes ago                            stupefied_rubin
b79e6fc69cab        httpd               "httpd-foreground"       17 minutes ago      Up 8 minutes                    80/tcp              myhttpd
1ec1c9643346        ee301c921b8a        "/hello"                 28 minutes ago      Exited (0) 28 minutes ago                           cocky_ramanujan
[root@ecs-docker ~]# docker logs a9b5ee00d73d
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
hello world
```

某些时候需要进入容器进行操作，可以使用 docker attach 命令或 docker exec 命令。

```
[root@ecs-docker ~]# docker run -dit ubuntu:14.04
c87f2f1ea058f4b5ca1238b31c3edfe2f43ab98268e25c88789e32ea4abf3f0e
[root@ecs-docker ~]# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS               NAMES
c87f2f1ea058        ubuntu:14.04        "/bin/bash"              6 seconds ago       Up 5 seconds                            friendly_pare
a9b5ee00d73d        ubuntu:14.04        "/bin/sh -c 'while t…"   About a minute ago  Up About a minute                       unruffled_thompson
b79e6fc69cab        httpd               "httpd-foreground"       18 minutes ago      Up 9 minutes        80/tcp              myhttpd
[root@ecs-docker ~]# docker attach c87f2f1ea058
root@c87f2f1ea058:/# ps
    PID TTY          TIME CMD
      1 pts/0    00:00:00 bash
     17 pts/0    00:00:00 ps
root@c87f2f1ea058:/#
```

删除容器

使用 docker rm 来删除一个处于终止状态的容器。若容器没有退出则无法删除，需要先停止容器。同镜像，添加-f 强制删除



四．私有仓库的搭建

安装运行 docker-registry

获取官方 registry 镜像并运行容器。



先在本机查看已有的镜像。此实验以 ubuntu:14.04 为例。



以 ubuntu:14.04 基础镜像为例，通过 docker tag 命令将基础镜像 ubuntu:14.04 镜像进行标记。

```
[root@ecs-docker ~]# docker tag ubuntu:14.04 127.0.0.1:5000/myubuntu:14.04
[root@ecs-docker ~]# docker images
REPOSITORY                TAG                 IMAGE ID            CREATED             SIZE
httpd                     latest              73acb239a8eb        4 weeks ago         177MB
registry                  latest              27031dfcd859        7 months ago        24.9MB
127.0.0.1:5000/myubuntu   14.04               55b7b4f7c5d6        18 months ago       187MB
ubuntu                    14.04               55b7b4f7c5d6        18 months ago       187MB
[root@ecs-docker ~]#
```

使用 docker push 上传标记的镜像

```
[root@ecs-docker ~]# docker push 127.0.0.1:5000/myubuntu:14.04
The push refers to repository [127.0.0.1:5000/myubuntu]
000e628b3e71: Pushed
926ca971b512: Pushed
59199d90878e: Pushed
14.04: digest: sha256:5ed16aa332467821529d451800e6fe599d83e30471e91b096752f8696d9bf6e9 size: 945
```

用 curl 查看仓库中的镜像

```
[root@ecs-docker ~]# curl 127.0.0.1:5000/v2/_catalog
{"repositories":["myubuntu"]}
```

删除已有镜像，再尝试从私有仓库中下载这个镜像

```
[root@ecs-docker ~]# docker image rm 127.0.0.1:5000/myubuntu:14.04
Untagged: 127.0.0.1:5000/myubuntu:14.04
Untagged: 127.0.0.1:5000/myubuntu@sha256:5ed16aa332467821529d451800e6fe599d83e30471e91b096752f8696d9bf6e9
[root@ecs-docker ~]# docker pull 127.0.0.1:5000/myubuntu:14.04
14.04: Pulling from myubuntu
Digest: sha256:5ed16aa332467821529d451800e6fe599d83e30471e91b096752f8696d9bf6e9
Status: Downloaded newer image for 127.0.0.1:5000/myubuntu:14.04
[root@ecs-docker ~]# docker images
REPOSITORY                TAG                 IMAGE ID            CREATED             SIZE
httpd                     latest              73acb239a8eb        4 weeks ago         177MB
registry                  latest              27031dfcd859        7 months ago        24.9MB
127.0.0.1:5000/myubuntu   14.04               55b7b4f7c5d6        18 months ago       187MB
ubuntu                    14.04               55b7b4f7c5d6        18 months ago       187MB
[root@ecs-docker ~]#
```

实验二 Dockerfile 的基本操作

五．Dockerfile 文件构建

下载基础镜像 centos:7

```
[root@ecs-docker ~]# docker pull centos:7
7: Pulling from library/centos
6717b8ec66cd: Pull complete
Digest: sha256:be65f488b7764ad3638f236b7b515b3678369a5124c47b8d32916d6487418ea4
Status: Downloaded newer image for centos:7
[root@ecs-docker ~]# docker images
REPOSITORY                TAG                 IMAGE ID            CREATED             SIZE
httpd                     latest              73acb239a8eb        4 weeks ago         177MB
registry                  latest              27031dfcd859        7 months ago        24.9MB
127.0.0.1:5000/myubuntu   14.04               55b7b4f7c5d6        18 months ago       187MB
ubuntu                    14.04               55b7b4f7c5d6        18 months ago       187MB
centos                    7                   c9a1fdca3387        2 years ago         301MB
```

创建 nginx_demo 文件夹

```
[root@ecs-docker ~]# pwd
/root
[root@ecs-docker ~]# mkdir nginx_demo
[root@ecs-docker ~]# ls
nginx_demo
[root@ecs-docker ~]#
```

进入文件夹，下载 nginx 源码压缩包

```
[root@ecs-docker ~]# cd nginx_demo
[root@ecs-docker nginx_demo]# wget http://nginx.org/download/nginx-1.12.2.tar.gz
--2024-05-07 16:19:03--  http://nginx.org/download/nginx-1.12.2.tar.gz
Resolving nginx.org (nginx.org)... 52.58.199.22, 3.125.197.172, 2a05:d014:5c0:2600::6, ...
Connecting to nginx.org (nginx.org)|52.58.199.22|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 981687 (959K) [application/octet-stream]
Saving to: 'nginx-1.12.2.tar.gz'

nginx-1.12.2.tar.gz                 100%[=======================================================================================>] 958.68K  1.26MB/s    in 0.7s

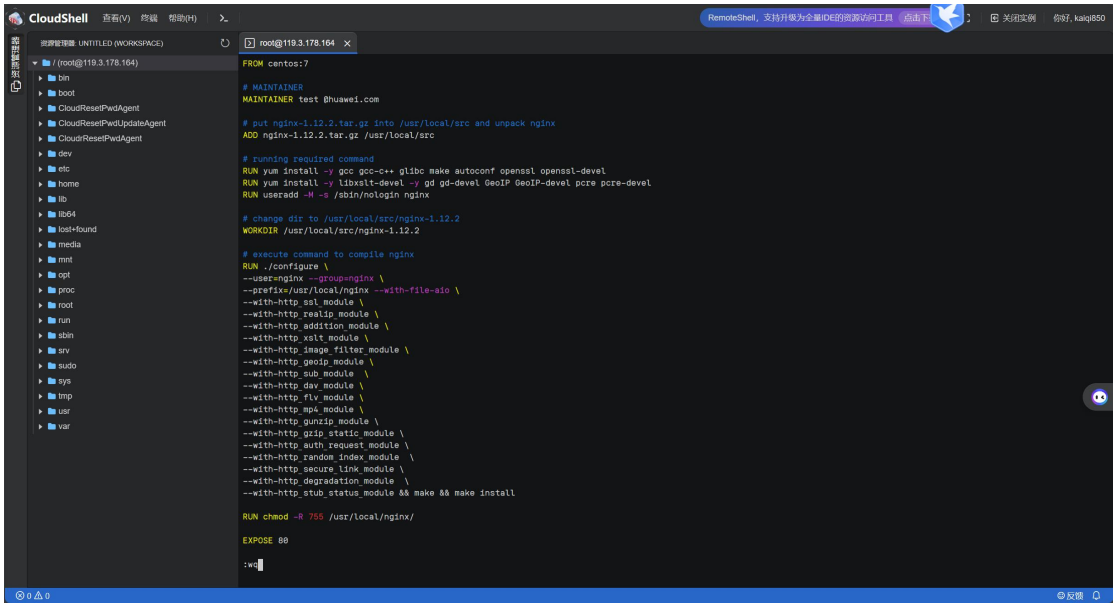
2024-05-07 16:19:04 (1.26 MB/s) - 'nginx-1.12.2.tar.gz' saved [981687/981687]

[root@ecs-docker nginx_demo]# --2020-08-xx 11:40:38--  http://nginx.org/download/nginx-1.12.2.tar.gz
```

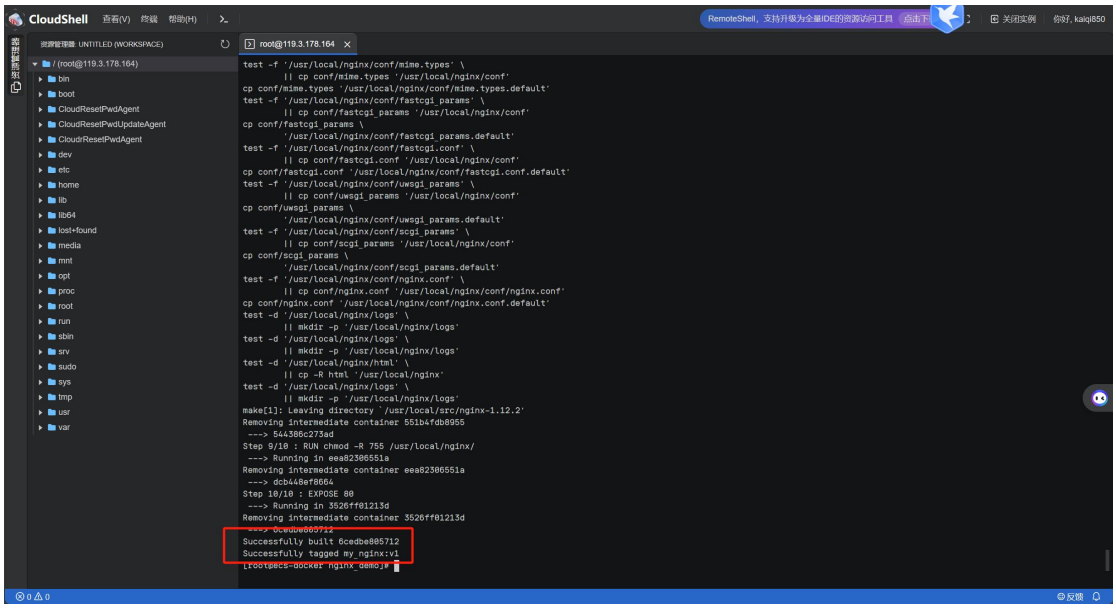
创建 Dockerfile 文件

```
[root@ecs-docker nginx_demo]# vim Dockerfile
```

输入 i，编辑如下内容到 Dockerfile 中



通过 Dockerfile 创建 nginx 镜像



查看构建的镜像

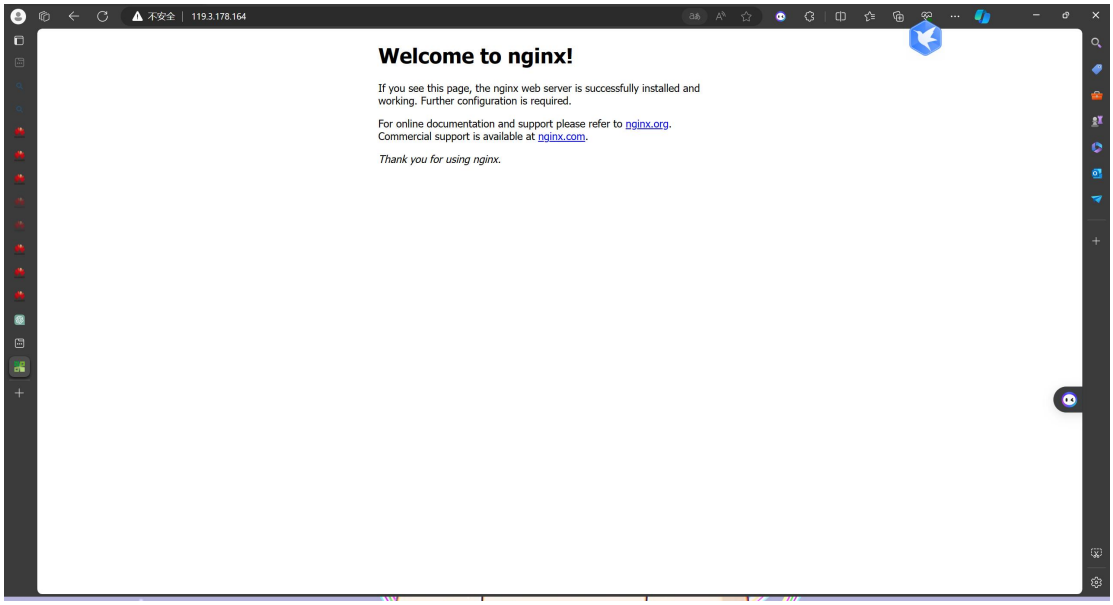
```
[root@ecs-docker nginx_demo]# docker images
REPOSITORY                TAG                 IMAGE ID            CREATED             SIZE
my_nginx                  v1                  6cedbe805712        37 seconds ago      719MB
httpd                     latest              73acb239a8eb        4 weeks ago         177MB
registry                  latest              27031dfcd859        7 months ago        24.9MB
127.0.0.1:5000/myubuntu   14.04               55b7b4f7c5d6        18 months ago       187MB
ubuntu                    14.04               55b7b4f7c5d6        18 months ago       187MB
centos                    7                   c9a1fdca3387        2 years ago         301MB
[root@ecs-docker nginx_demo]#
```

六. Nginx 镜像验证

通过构建的镜像，运行一个容器，将端口进行映射。

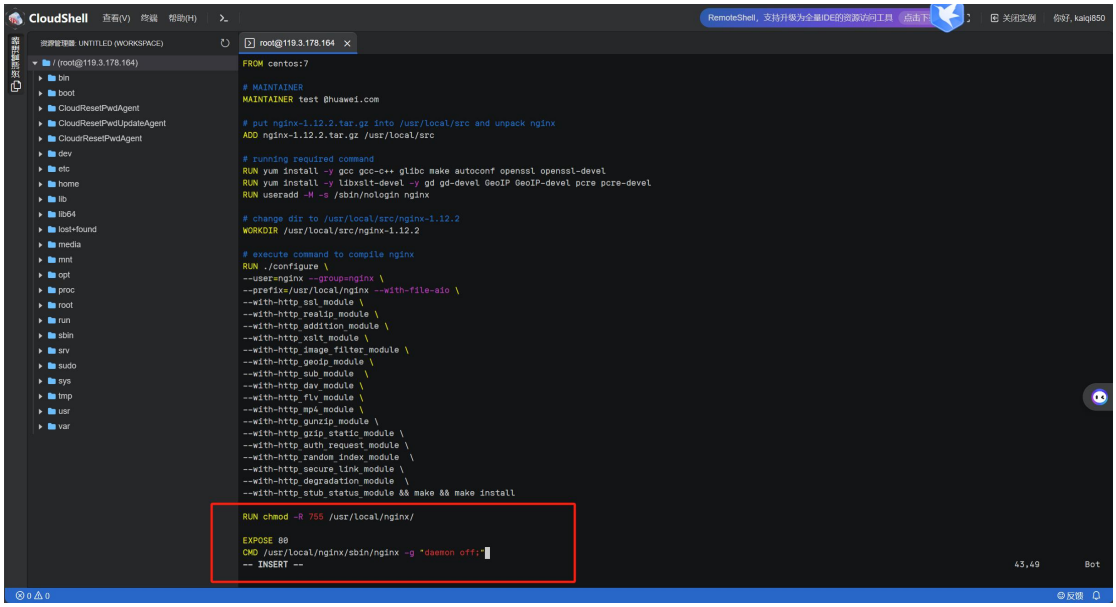
```
[root@ecs-docker nginx_demo]# docker run -d -p 80:80 my_nginx:v1 /usr/local/nginx/sbin/nginx -g "daemon off;"
4bb5a96147bfdc0ffe91ecda185f8a1032c9fd2f1b1da6ce9bd5fec410e7e3e0
[root@ecs-docker nginx_demo]#
```

打开浏览器，输入 ecs-docker 弹性 IP 地址，默认端口为 80，进行验证，显示“Welcome to nginx!”，说明容器运行正常。

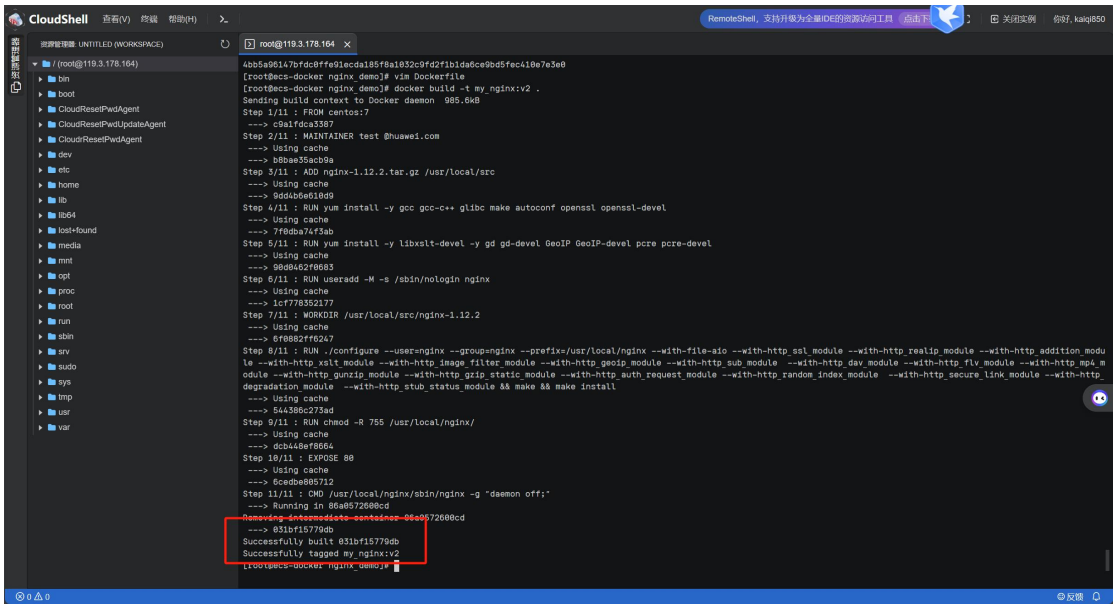


我们也可以基于以上 Dockerfile 文件依次添加其他的指令进行构建，比如我们可以添加 CMD 命令，设置 nginx 非 daemon 守护进程，这样容器启动时不会自动退出。

创建 Dockerfile 文件,输入 i 进入编辑模式，在原有 Dockerfile 基础上，增加如下内容到 Dockerfile 最后一行。



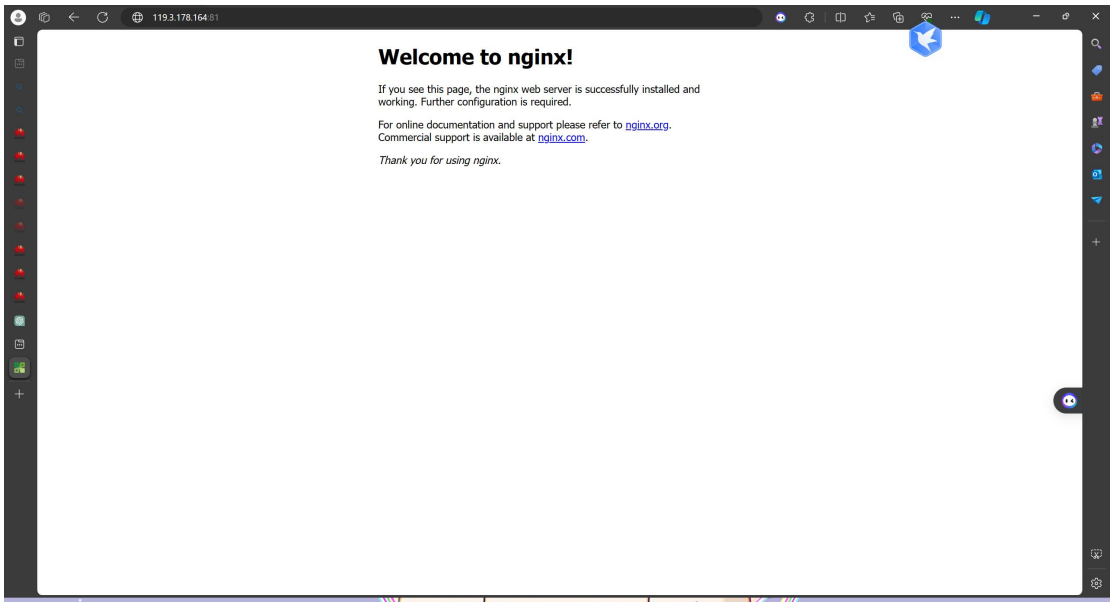
通过 Dockerfile 创建新的 nginx 镜像



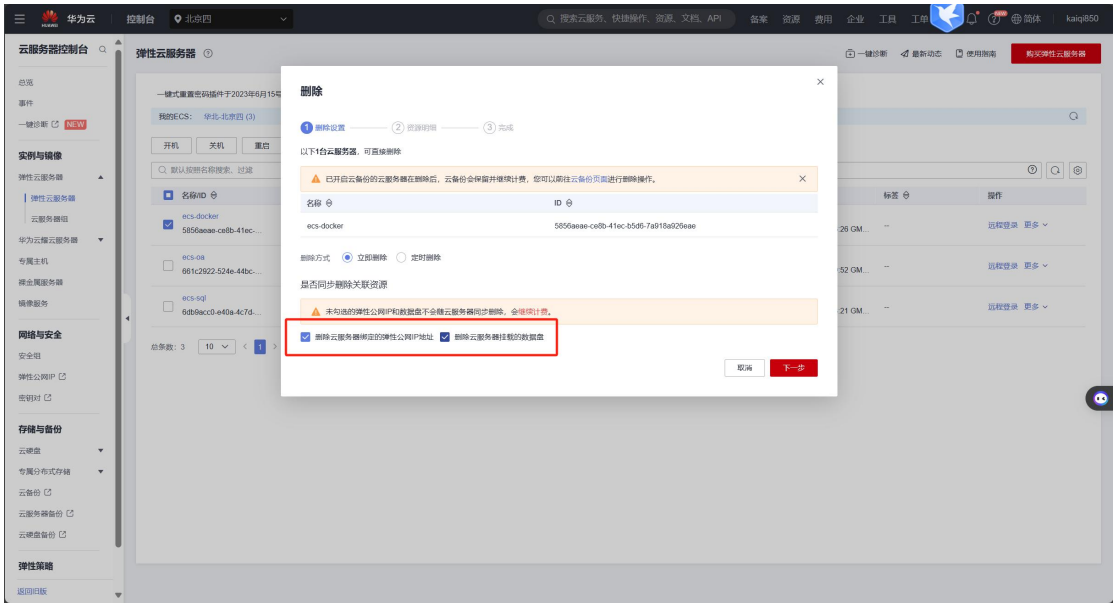
通过构建的镜像，运行一个容器，将端口进行映射，将容器的 80 端口映射到主机的 81 端口。

```
[root@ecs-docker nginx_demo]# docker run -d -p 81:80 my_nginx:v2
b85ab767aa68133c3f08d581b12019a1d951c2bb324f0af0e7bb606c5ecf4d49
[root@ecs-docker nginx_demo]# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS                    NAMES
b85ab767aa68        my_nginx:v2         "/bin/sh -c '/usr/lo…"   15 seconds ago      Up 15 seconds       0.0.0.0:81->80/tcp       musing_goldberg
4bb5a96147bf        my_nginx:v1         "/usr/local/nginx/sb…"   4 minutes ago       Up 4 minutes        0.0.0.0:80->80/tcp       amazing_bhaskara
1aa5e18ba230        registry            "/entrypoint.sh /etc…"   25 minutes ago      Up 25 minutes       0.0.0.0:5000->5000/tcp   registry
a9b5ee00d73d        ubuntu:14.04        "/bin/sh -c 'while t…"   34 minutes ago      Up 34 minutes                                unruffled_thompson
b79e6fc69cab        httpd               "httpd-foreground"       About an hour ago   Up 42 minutes       80/tcp                   myhttpd
[root@ecs-docker nginx_demo]#
```

打开浏览器进行验证，打开浏览器，输入弹性 IP 地址，端口为 81，进行验证，显示“Welcome to nginx!”，说明容器运行正常。



释放资源



实验三 鲲鹏平台 OA 系统容器化部署

七．通过 Dockerfile 构建 OA 系统镜像

步骤 1 使用《鲲鹏云上应用高可用部署》实验中所创建的 OA 应用服务器，如 ecs-oa 或 ecs-oa-kunpeng。远程登录 ECS，检查内核版本。

```
[root@ecs-oa ~]# uname -r
4.19.90-2110.8.0.0119.oe1.aarch64
[root@ecs-oa ~]#
```

若安装过 docker，需要先删掉，之后再安装依赖

```
[root@ecs-oa ~]# uname -r
4.19.90-2110.8.0.0119.oe1.aarch64
[root@ecs-oa ~]# yum remove docker docker-common docker-selinux docker-engine
No match for argument: docker
No match for argument: docker-common
No match for argument: docker-selinux
No match for argument: docker-engine
No packages marked for removal.
Dependencies resolved.
Nothing to do.
Complete!
[root@ecs-oa ~]# yum install -y device-mapper-persistent-data lvm2
Last metadata expiration check: 3:28:39 ago on Tue 07 May 2024 01:19:32 PM CST.
Dependencies resolved.
================================================================================
 Package                  Architecture        Version                 Repository      Size
================================================================================
Installing:
 lvm2                     aarch64             8:2.02.187-3.oe1        update         1.5 M
 thin-provisioning-tools  aarch64             0.7.6-6.oe1             update         353 k
Installing dependencies:
 device-mapper-event      aarch64             8:1.02.170-3.oe1        update          42 k
 libaio                   aarch64             0.3.112-1.oe1           update          22 k

Transaction Summary
================================================================================
Install  4 Packages
```

启动 Docker 后台服务，并测试运行 hello-world

```
[root@ecs-oa ~]# systemctl start docker
[root@ecs-oa ~]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
478afc919002: Pull complete
Digest: sha256:a26bff933ddc26d5cdf7faa98b4ae1e3ec20c4985e6f87ac0973052224d24302
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

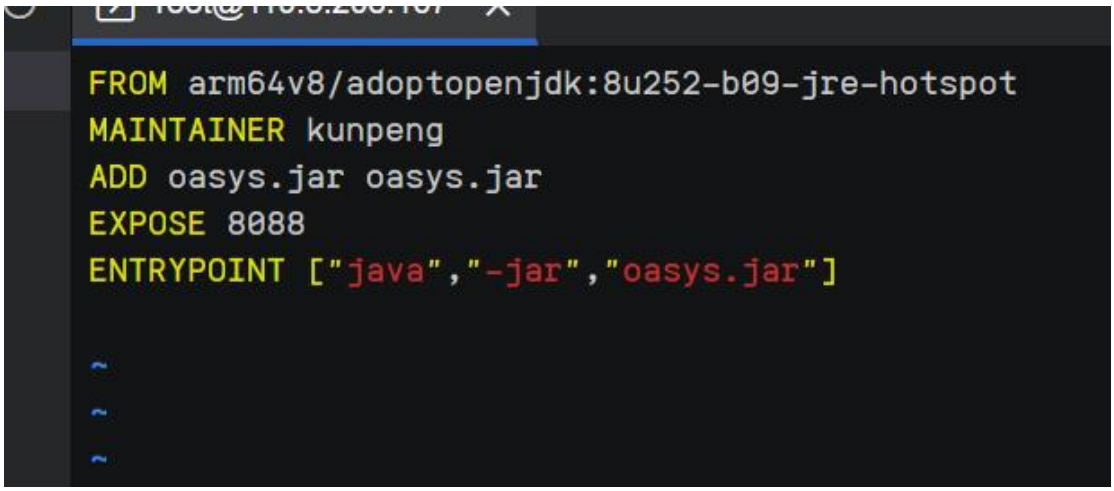
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

进入目录，创建Dockerfile，按"i"进入编辑模式，输入如下内容到Dockerfile中。完成后点击“Esc”键，然后输入“:wq”后按回车键，保存退出文档



```
FROM arm64v8/adoptopenjdk:8u252-b09-jre-hotspot
MAINTAINER kunpeng
ADD oasys.jar oasys.jar
EXPOSE 8088
ENTRYPOINT ["java","-jar","oasys.jar"]
~
~
~
```

配置镜像加速器，通过以下操作修改daemon配置文“/etc/docker/daemon.json”来使用加速器。

```
[root@ecs-oa target]# mkdir -p /etc/docker
[root@ecs-oa target]# tee /etc/docker/daemon.json <<-'EOF'
>
> {
> "registry-mirrors": ["https://0961a4605080f5660f12c014b740c160.mirror.swr.myhuaweicloud.com"]
> }
>
> EOF

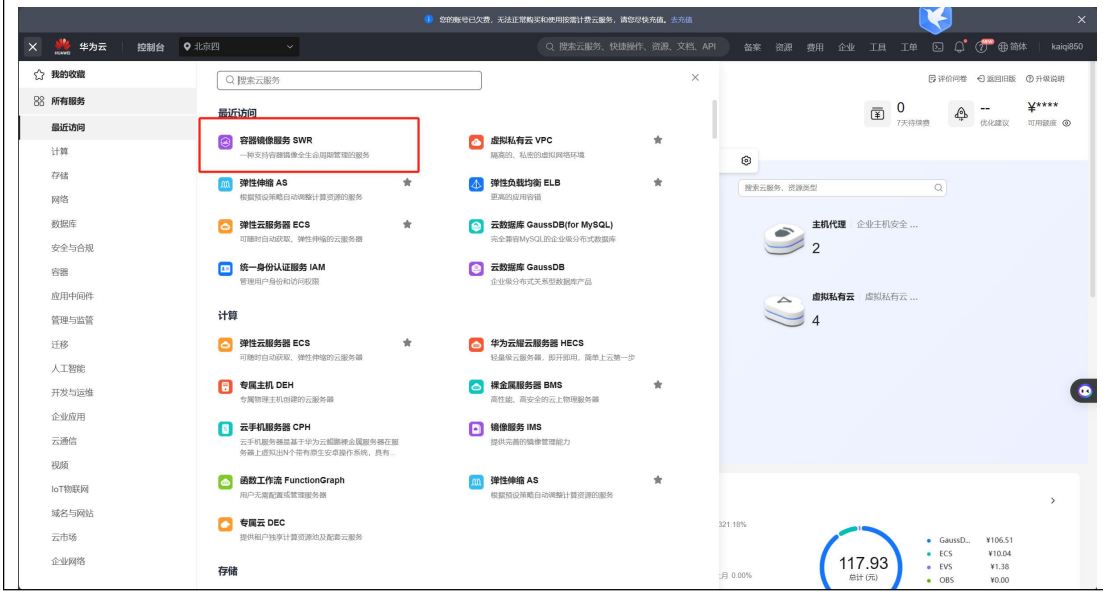
{
"registry-mirrors": ["https://0961a4605080f5660f12c014b740c160.mirror.swr.myhuaweicloud.com"]
}

[root@ecs-oa target]#
[root@ecs-oa target]# systemctl daemon-reload
[root@ecs-oa target]# systemctl restart docker
```

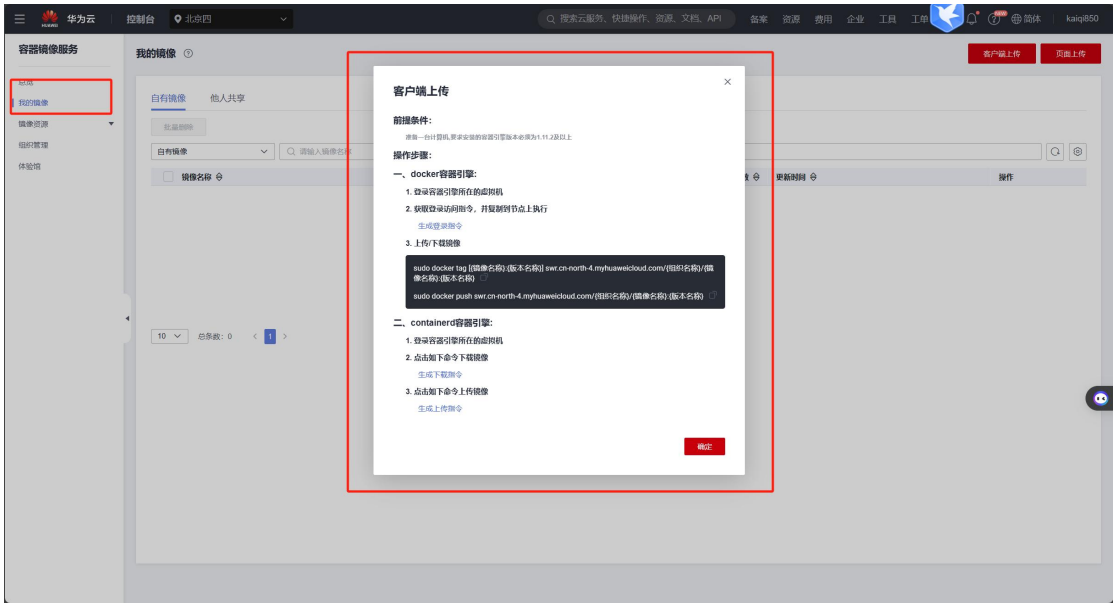
通过 Dockerfile 创建 OA 系统镜像，镜像名为“oa-kunpeng”，标签为 1.0 。

```
[root@ecs-oa target]# docker build -t oa-kunpeng:1.0 .
Sending build context to Docker daemon  58.71MB
Step 1/5 : FROM arm64v8/adoptopenjdk:8u252-b09-jre-hotspot
8u252-b09-jre-hotspot: Pulling from arm64v8/adoptopenjdk
68fe03cb170d: Pull complete
18959295effb: Pull complete
51118fb70ce3: Pull complete
409a5d9eae93: Pull complete
5427b5697c73: Pull complete
9a7d0a6d194a: Pull complete
Digest: sha256:0519885cf32b031c972b18f9bb0d41487beddd8acd5aa4c88b6156164eeb944a
Status: Downloaded newer image for arm64v8/adoptopenjdk:8u252-b09-jre-hotspot
 ---> db880952cdcf
Step 2/5 : MAINTAINER kunpeng
 ---> Running in 24089f65524f
Removing intermediate container 24089f65524f
 ---> d5410f18ea85
Step 3/5 : ADD oasys.jar oasys.jar
 ---> 0659c512ad3f
Step 4/5 : EXPOSE 8088
 ---> Running in 6a295f92b315
Removing intermediate container 6a295f92b315
 ---> f9599b37f842
Step 5/5 : ENTRYPOINT ["java","-jar","oasys.jar"]
 ---> Running in a8a4137df137
Removing intermediate container a8a4137df137
 ---> 97be1cde9efe
Successfully built 97be1cde9efe
Successfully tagged oa-kunpeng:1.0
[root@ecs-oa target]# docker images
REPOSITORY              TAG                 IMAGE ID            CREATED             SIZE
oa-kunpeng              1.0                 97be1cde9efe        10 seconds ago      241MB
hello-world             latest              ee301c921b8a        12 months ago       9.14kB
arm64v8/adoptopenjdk    8u252-b09-jre-hotspot   db880952cdcf        3 years ago         199MB
```

登录华为云官网,选择“产品 -> 基础服务 -> 容器服务 -> 容器镜像服务 SWR”，然后点击立即使用。



进入容器镜像服务页面，点击“我的镜像 -> 客户端上传”。



将 docker login 指令复制到 ECS 下的 docker 环境中。显示 login Succeeded，说明登录成功。



```
[root@ecs-oa target]# docker login -u cn-north-4@Z93DULI4BL5SQK2QMDLQ -p 4bf4deb02f4261accbcb99c5671add770f029fe42e8da93536e93158421c600f swr.cn-north-4.myhuaweicloud.com
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

对要上传的镜像进行命名打标签，其中镜像名称和版本名称为之前设置的 oa-kunpeng:1.0，组织名称为第 3 步所创建的。

```
[root@ecs-oa target]# docker tag oa-kunpeng:1.0 swr.cn-north-4.myhuaweicloud.com/kaiqi850/oa-kunpeng:1.0
[root@ecs-oa target]#
```

将打完标签的镜像上传

```
[root@ecs-oa target]# docker tag oa-kunpeng:1.0 swr.cn-north-4.myhuaweicloud.com/kaiqi850/oa-kunpeng:1.0
[root@ecs-oa target]# docker push swr.cn-north-4.myhuaweicloud.com/kaiqi850/oa-kunpeng:1.0
The push refers to repository [swr.cn-north-4.myhuaweicloud.com/kaiqi850/oa-kunpeng]
291a1e7e3699: Pushed
a92373e02ea9: Pushed
a28a4c546980: Pushed
04d7cd174b7a: Pushed
119869ffe539: Pushed
1fabb61d4382: Pushed
8b97a1c69eda: Pushed
1.0: digest: sha256:b0885f2ddc8baea45aeacba9a6782ad53f7a75be1c4118bad6251123643c0b39 size: 1788
```

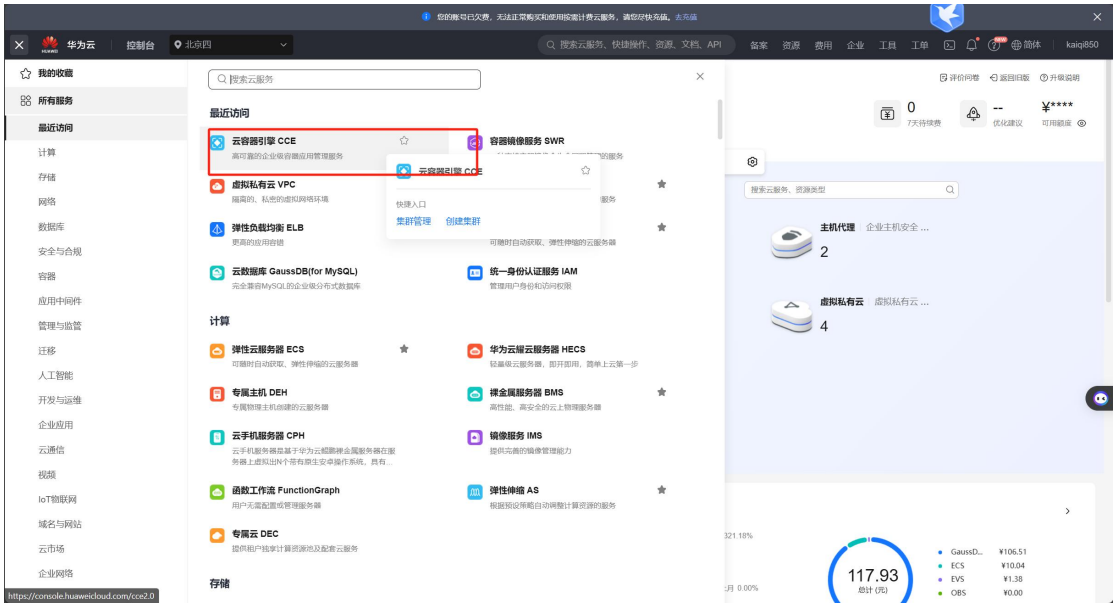
返回容器镜像服务页面，点击“我的镜像”，可以看到镜像已经上传成功。

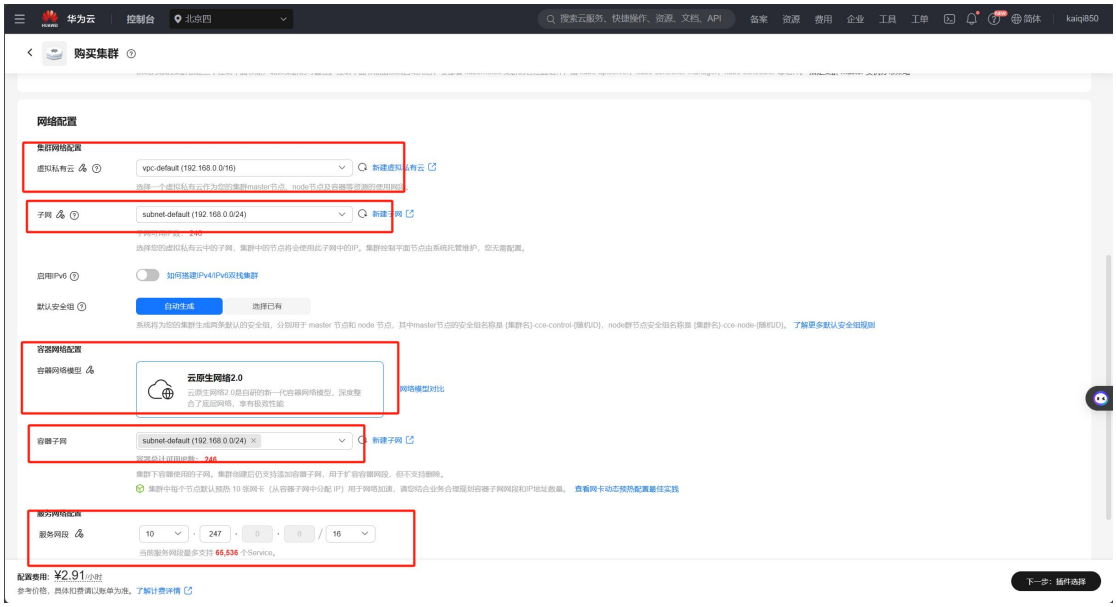


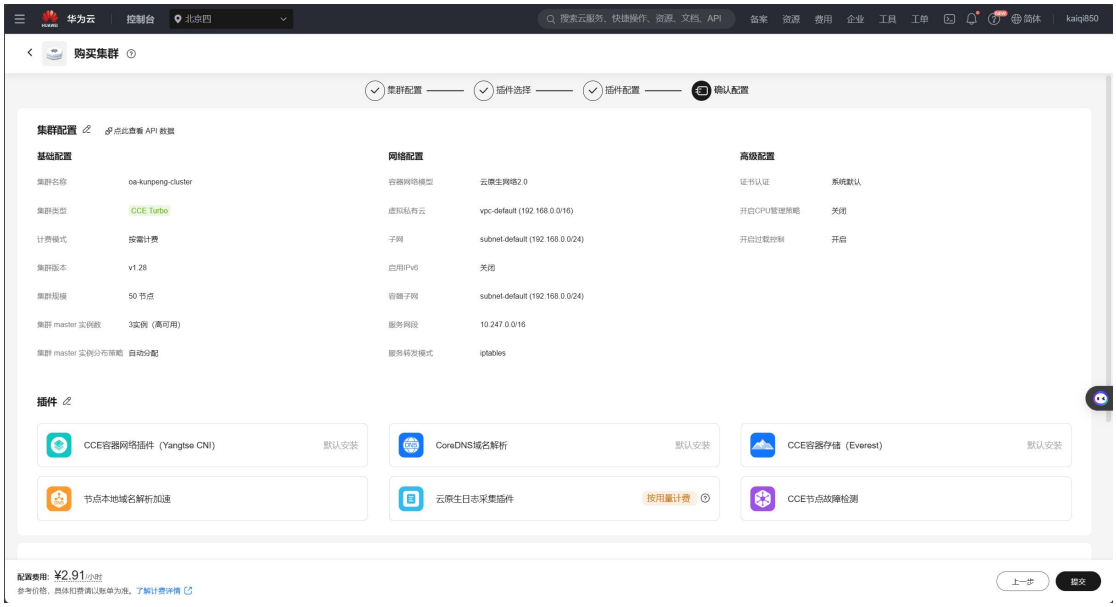
八．CCE 鲲鹏集群创建

创建 CCE 集群

分别点击“产品 -> 基础服务 -> 云容器引擎 CCE”，并按照以下参数设置 CCE 集群服务选项信息。

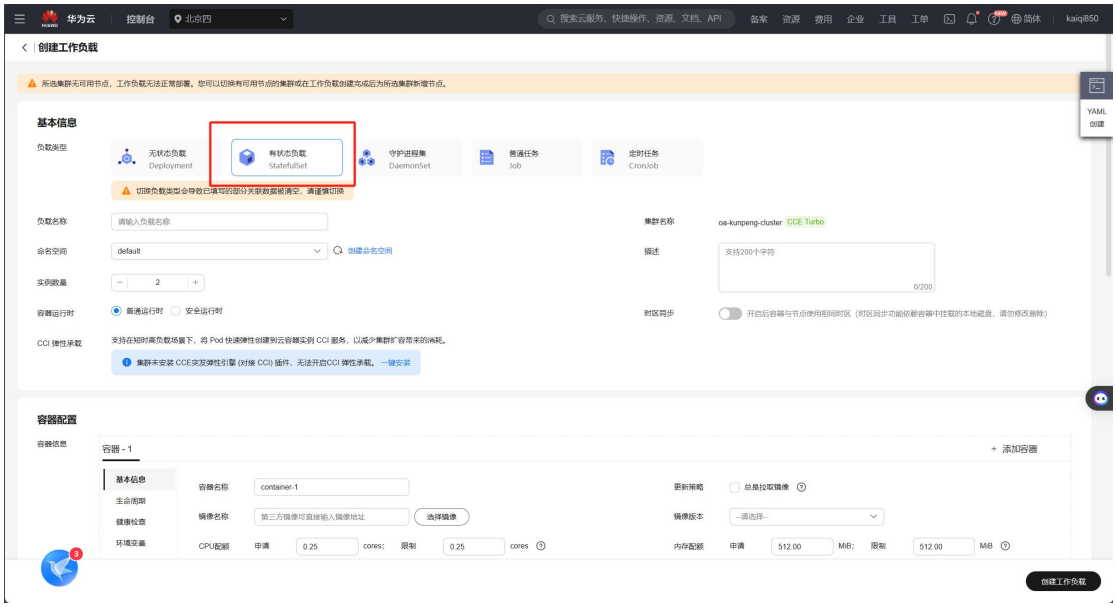






九．CCE 部署 OA 系统

返回云容器引擎页面，分别单击“工作负载 -> 有状态负载（StatefulSet） -> 创建有状态工作负载”。



按照以下参数配置设置工作负载基本信息



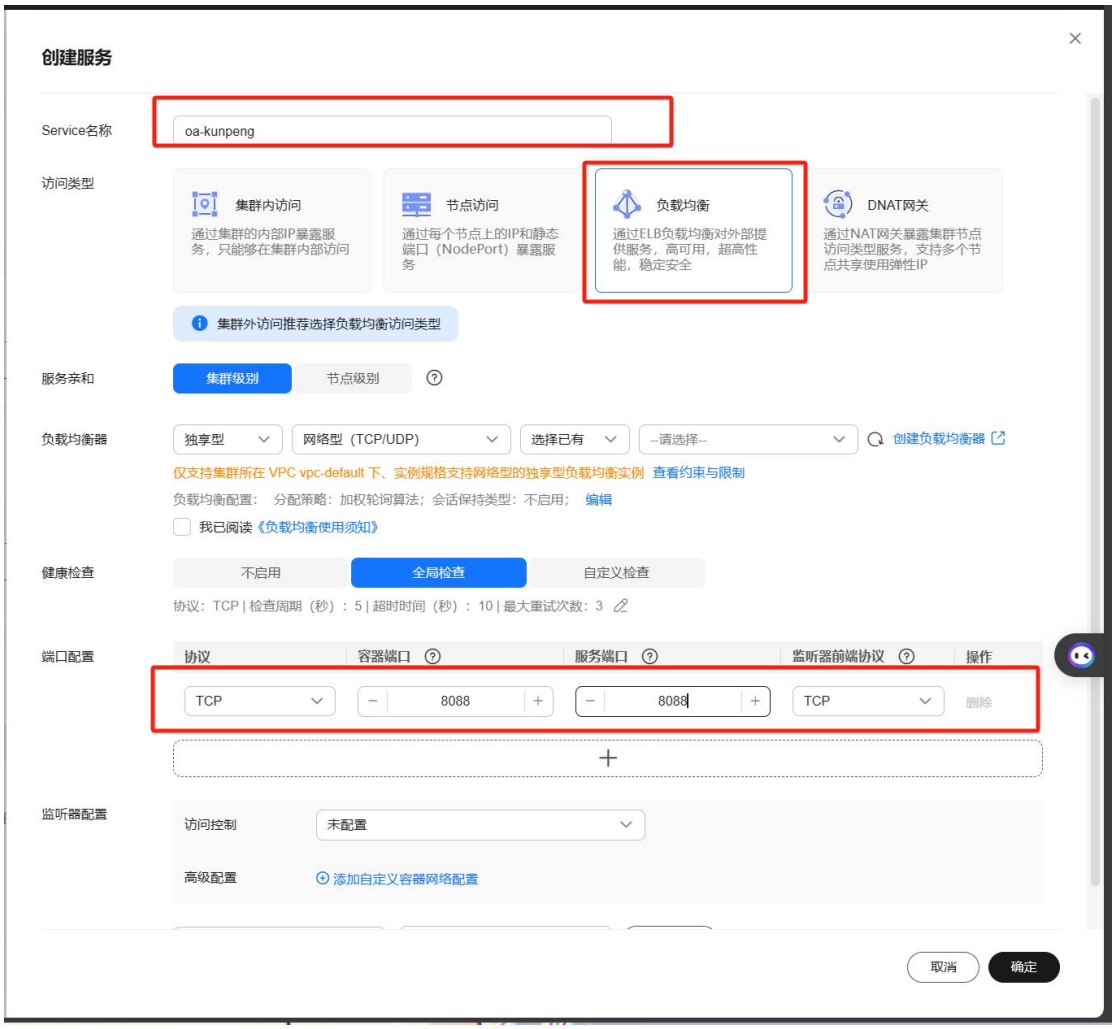
选择“添加容器”，点击“我的镜像”，然后选择之前上传至的“oa-kunpeng”镜像



设置容器端口名称为：port ，端口为：8088



点击“添加服务”，选择如下参数



待“oa-kunpeng”实例执行状态为安装成功后，单击“访问方式”，复制访问地址中的弹性公网 IP。通过弹性公网 IP 加 8088 端口访问 OA 系统登录页，使用用户名“soli”和密码“123456”登录进入 OA 系统主界面





十．释放资源

删除云数据库 GaussDB



删除 CCE 鲲鹏集群



删除 ECS 弹性云服务器



删除 IMS 镜像

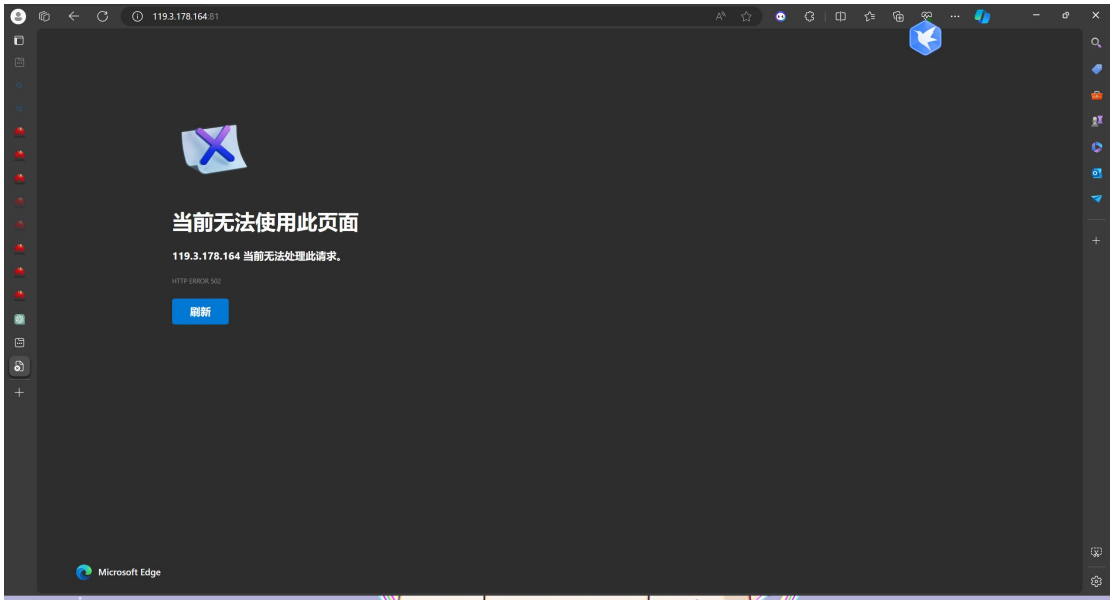


五、实验数据（现象）处理分析

问题与解决：

Q1：已经成功将容器与主机的端口进行映射，但仍无法访问对应页面





S1：检查发现挂载了代理，导致本机的网络环境发生了变化，后重新设置了网络代理，恢复正常

六、实验结论

通过本次实验，成功掌握了 Docker 的核心操作流程，包括在华为云 ECS 实例上安装配置 Docker、镜像的拉取与管理（如 **nginx**、**ubuntu:14.04**）、容器的生命周期控制（启停、进入、删除）以及私有镜像仓库的搭建与验证。实验过程中，通过 **docker run** 命令验证了 **hello-world** 镜像的运行，并通过本地 **docker-registry** 实现了镜像的推送（**docker push**）与拉取（**docker pull**），验证了私有仓库的可用性。此外，安全组配置与端口放通（如 5000 端口）的实践，加深了对云网络环境管理的理解。

在 Dockerfile 构建实验中，通过编写自定义 Dockerfile，从 **centos:7** 基础镜像编

译安装 Nginx 1.12.2，掌握了 **FROM**、**ADD**、**RUN**、**EXPOSE** 等指令的应用。通过多阶段构建优化镜像体积，并添加 **CMD** 指令确保容器以非守护进程模式运行，解决了容器自动退出的问题。实验验证了端口映射（**-p 80:80**）的有效性，成功通过浏览器访问 Nginx 默认页面，体现了从源码到服务的完整构建流程。此部分实验不仅提升了镜像定制能力，还强化了服务部署的规范性。

在 OA 系统容器化部署中，成功将 Java 应用打包为 Docker 镜像，并集成华为云 SWR 与 CCE 服务。通过 Dockerfile 定义 JDK 环境与 JAR 包依赖，利用 SWR 实现镜像托管，并在 CCE 中创建鲲鹏集群完成有状态工作负载的部署。最终通过弹性公网 IP 绑定，实现了 OA 系统的外部访问（**http://<IP>:8088**），验证了容器化应用的高可用性。实验全程强调资源管理，最终通过系统化的资源释放操作（删除 ECS、CCE 集群、SWR 镜像等），避免了云资源浪费。此实验不仅巩固了容器化技术，还深化了对云原生架构与企业级运维流程的理解。

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：